WEEK 2 · LESSON 11 OF 24 · HANDOUT

Datasets, DataLoaders and Tabular Data

Study time: **about 45 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=V_xro1bcAuA — open the lesson online to watch inside the course.

🎓 **Go deeper:** EuroPython: How to train an image classifier using PyTorch — https://www.youtube.com/watch?v=eDRck258bnw

Your goal in this lesson: Feed data to PyTorch with `Dataset` and `DataLoader`, prepare tabular data correctly, and split data into training, validation and test sets.

Why not just slice tensors?

Until now we shuffled indices by hand. Real projects use PyTorch's two data tools:

- **Dataset** — knows how many examples there are and how to get example number i.
- **DataLoader** — wraps a Dataset and gives you shuffled **mini-batches**, optionally loading them in parallel.

Images that do not fit in memory, text files, audio — all are handled the same way: write a Dataset, wrap it in a DataLoader, and the training loop never changes.

The loans data again

We use the mobile-money micro-loans data from Machine Learning Fundamentals: predict `defaulted` (1 = did not repay).

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(11)
n = 2000
income = rng.lognormal(12.2, 0.45, n).round(-3)
loan = rng.uniform(20_000, 300_000, n).round(-3)
months = rng.integers(1, 61, n)
late = rng.poisson(0.6, n)
salary = rng.random(n) < 0.55
z = -3.2 + 2.5 * (loan / income) - 0.03 * months + 0.8 * late - 0.9 * salary
default = (rng.random(n) < 1 / (1 + np.exp(-z))).astype(int)
loans = pd.DataFrame({"income_rwf": income, "loan_rwf": loan, "months_customer": months,
                      "late_payments": late, "salaried": salary.astype(int),
                      "defaulted": default})
print(loans.shape, "default rate:", round(loans["defaulted"].mean(), 3))
loans.head()
```

Prepare tabular data for a network

Networks are as sensitive to scale as k-NN: income in hundreds of thousands next to a 0/1 column would make training slow and unstable. **Standardise**, using the training rows only:

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = loans.drop(columns="defaulted").to_numpy()
y = loans["defaulted"].to_numpy()
X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)
X_train, X_val, y_train, y_val = train_test_split(X_dev, y_dev, test_size=0.2, random_state=42,
stratify=y_dev)

scaler = StandardScaler().fit(X_train)            # learn from training rows only
X_train, X_val, X_test = (scaler.transform(a) for a in (X_train, X_val, X_test))
print(len(X_train), len(X_val), len(X_test))
```

A custom Dataset

```
import torch
from torch.utils.data import Dataset, DataLoader

class LoanDataset(Dataset):
    def __init__(self, features, labels):
        self.x = torch.tensor(features, dtype=torch.float32)
        self.y = torch.tensor(labels, dtype=torch.float32)

    def __len__(self):
        return len(self.y)

    def __getitem__(self, i):
        return self.x[i], self.y[i]

train_ds = LoanDataset(X_train, y_train)
print(len(train_ds), train_ds[0])
```

Only two methods are needed: `__len__` and `__getitem__`.

A DataLoader

```
train_loader = DataLoader(train_ds, batch_size=64, shuffle=True)
val_loader = DataLoader(LoanDataset(X_val, y_val), batch_size=256)

xb, yb = next(iter(train_loader))
print(xb.shape, yb.shape)          # one batch: 64 rows of 5 features
print("batches per epoch:", len(train_loader))
```

- `shuffle=True` for training (a new order every epoch); no shuffling for validation and test.
- `num_workers=2` loads batches in parallel — useful for images on disk.
- For data already in tensors, `TensorDataset(x, y)` saves writing a class.

Train with the loader

```
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(0)
model = nn.Sequential(nn.Linear(5, 32), nn.ReLU(), nn.Linear(32, 1))
opt = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(30):
    model.train()
    for xb, yb in train_loader:                # the loop is now very clean
        opt.zero_grad()
        loss = F.binary_cross_entropy_with_logits(model(xb).squeeze(1), yb)
        loss.backward()
        opt.step()

model.eval()
with torch.no_grad():
    val_logits = torch.cat([model(xb).squeeze(1) for xb, _ in val_loader])
val_acc = ((val_logits > 0).float().numpy() == y_val).mean()
print("validation accuracy:", round(val_acc, 3))
```

Categories in tabular networks

For a column such as district, either one-hot encode it (as in Machine Learning Fundamentals) or use an **embedding** layer that learns a small vector for each category — the same idea used for words in Week 4.

Should you use a network for tables?

Honest advice from industry and research: on ordinary tables of a few thousand rows, **gradient boosting and logistic regression are usually as good or better** than neural networks, and they are faster and easier to explain. Networks shine with images, sound, text and very large or mixed datasets. The next lesson's project tests this claim.

Key terms

- **Dataset** — gives the number of examples and example i.
- **DataLoader** — turns a Dataset into shuffled mini-batches.
- **TensorDataset** — a ready-made Dataset for tensors.
- **Batch size** — examples per update.
- **Embedding** — a learned vector for each category.

Remember this: Wrap your data in a Dataset and a DataLoader, scale features with training statistics only, and the same clean training loop works for any data.

Name: Date:

Activity

Task 1

Replace `LoanDataset` with `TensorDataset` and check that training gives the same result.

Task 2

Train with batch sizes 8, 64 and 512. How do speed and validation accuracy change?

Task 3

Remove the StandardScaler and train again. What happens?

Task 4

Add the engineered feature `loan_to_income` from Machine Learning Fundamentals before scaling. Does the network improve?

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Which two methods must a custom Dataset have?

   A. fit and predict
   B. __len__ and __getitem__
   C. forward and backward
   D. train and eval

2. What does a DataLoader provide?

   A. Plots
   B. A trained model
   C. A loss function
   D. Shuffled mini-batches from a Dataset

3. Which loader should shuffle?

   A. Only the training loader
   B. Only the test loader
   C. Both validation and test
   D. Neither

4. Why standardise tabular features for a network?

   A. Networks cannot read decimals
   B. To add labels
   C. It is not needed
   D. Features with big numbers would make training slow and unstable

5. Where must the scaler learn its mean and standard deviation?

   A. From the labels
   B. From the training rows only
   C. From all the data
   D. From the test rows

6. For a small table of a few thousand rows, which is often as good or better than a neural network?

   A. A CNN
   B. A transformer
   C. An LSTM
   D. Gradient boosting or logistic regression

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.